

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Архитектура вычислительных систем

ОТЧЕТ
к лабораторной работе № 2
на тему «Мультизадачность в защищенном режиме»

Выполнил:
студент гр. 153503
Кахновский Е.С.

Проверил:
Калиновская А.А.

Минск 2023

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результаты выполнения лабораторной работы.....	5
Выводы	6
Список использованных источников	7
Приложение А (обязательное) Листинг кода.....	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью выполнения данной лабораторной работы является изучение принципов и средств реализации мультизадачности в защищенном режиме процессора.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Под мультизадачностью подразумевают способность компьютера выполнять несколько задач одновременно. На самом деле процессор некоторое время выполняет один командный поток, затем быстро переключается на второй и выполняет его, переключается на третий и т.д. При этом при каждом переключении сохраняется контекст прерываемого потока, так что потом процессор сможет "безболезненно" продолжить выполнение прерванного потока команд. Благодаря высокому быстродействию создается иллюзия того, что все задачи выполняются одновременно (параллельно).

Для управления мультизадачностью нет специальных команд. Задачи переключаются командами FAR CALL, FAR JMP, INT, IRET. Однако при этом участвуют специальные дескрипторы: дескриптор сегмента состояния задачи (Task State Segment) и дескриптор шлюза задачи. Когда управление передается на один из таких дескрипторов, происходит переключение задачи. При переключении задачи процессор сохраняет (восстанавливает) свой контекст в сегменте состояния задачи (TSS). Селектор TSS выполняемой задачи хранится в регистре задачи (Task Register). При переключении задачи процессор может сменить LDT, что позволяет назначить каждой задаче свое адресное пространство, недоступное для других задач. Можно также перегрузить CR3, что позволяет применить для изолирования задач механизм страничного преобразования.

Дескриптор TSS относится к системным дескрипторам. Поле Type дескриптора TSS может содержать код 1001, если это доступный TSS, или 1011, если это занятый TSS, т.е. если задача активна в настоящий момент.

На рис. 1 представлен формат сегмента TSS для процессора i80386. Из рисунка видно, что в TSS предусмотрены поля для хранения сегментных регистров GS, FS, DS, SS, CS, ES. Имеется поле для хранения содержимого регистра LDTR, указывающего на локальную таблицу дескрипторов, распределённую данной задаче. Для хранения содержимого 32-разрядных регистров используются поля TSS, обозначенные на рисунке как EDI, ESI, EBP, ESP, EBX, EDX, ECX, EAX, EFLAGS, EIP.

Поле CR3 хранит содержимое системного регистра CR3. Этот регистр является указателем на каталог таблиц страниц. Таким образом, каждая задача может иметь свой собственный каталог таблиц страниц, что позволяет выполнить изоляцию задач не только на уровне сегментов, но и на уровне страниц.[1]

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

В рамках выполнения лабораторной работы было создано приложение, которое переключает процессор в защищенный режим, а затем запускает на выполнение 2 задачи. Каждая задача выводит на экран свое сообщение. Первая задача выводит последовательность символов ASCII начиная с 48 символа, а вторая задача, начиная с 0. Когда все задачи отработают программа возвращает процессор в реальный режим. Результат работы программы представлен на (рисунок 3.1).

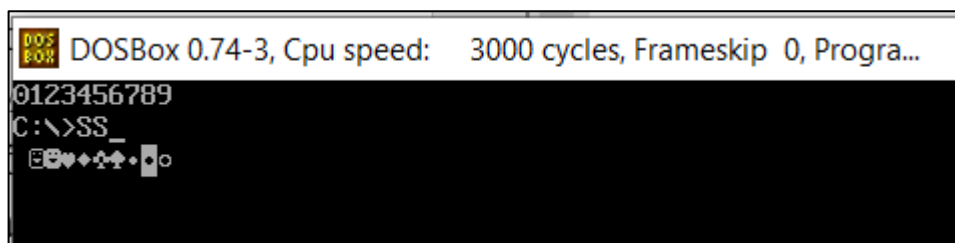


Рисунок 3.1 – Результат работы программы

ВЫВОДЫ

В результате выполнения этой лабораторной работы была разработана программа с использованием DOSBox, которая переключает процессор в защищенный режим, а затем запускает на выполнение 2 задачи. Каждая задача выводит на экран свое сообщение. Когда все задачи отработывают программа возвращает процессор в реальный режим.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Юрий А. Уровни защиты. – СПб: Питер, 2000. – 158 с.
- [2] [Электронный ресурс]. – Режим доступа:
https://dfe.petsu.ru/x86/env_1.shtml – Дата доступа 08.10.2023

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг 1 – Файл lab2.asm

```
.386p
RM_seg segment para public "CODE" use16
    assume cs:RM_seg,ds:PM_seg,ss:stack_seg ; Берем точку отсчёта для
инструкций со смещением к переменным.
start:
; подготовить сегментные регистры
    push    PM_seg
    pop     ds
; проверить, не находимся ли мы уже в PM
    mov     eax,cr0 ; Сохранили значение cr0
    test    al,1 ; Проверили на установку младшего бита
    jz      no_V86 ; Если 0 - продолжаем
; сообщить и выйти
    mov     dx,offset v86_msg ; загружаем в dx сообщение об ошибке
err_exit: ; Вывод сообщения
    push    cs
    pop     ds
    mov     ah,9
    int     21h
    mov     ah,4Ch
    int     21h

; убедиться, что мы не под Windows
no_V86:
    mov     ax,1600h
    int     2Fh
    test    al,al
    jz      no_windows
; сообщить и выйти
    mov     dx,offset win_msg
    jmp     short err_exit

; сообщения об ошибках при старте
v86_msg db    "Процессор в режиме V86 - нельзя переключиться в PM$"
win_msg db    "Программа запущена под Windows - нельзя перейти в кольцо 0$"

; итак, мы точно находимся в реальном режиме
no_windows:
; очистить экран
    mov     ax,3
    int     10h
; вычислить базы для всех дескрипторов сегментов данных
; xor     eax,eax
    mov     ax,RM_seg
    shl     eax,4
    mov     word ptr GDT_16bitCS+2,ax
    shr     eax,16
    mov     byte ptr GDT_16bitCS+4,al
```



```

        mov     ax,PM_seg
        shl     eax,4
        mov     word ptr GDT_32bitCS+2,ax
        mov     word ptr GDT_32bitSS+2,ax
        shr     eax,16
        mov     byte ptr GDT_32bitCS+4,al
        mov     byte ptr GDT_32bitSS+4,al
; вычислить линейный адрес GDT
        xor     eax,eax
        mov     ax,PM_seg
        shl     eax,4
        push    eax
        add     eax,offset GDT
        mov     dword ptr gdtr+2,eax
; загрузить GDT
        lgdt    fword ptr gdtr
; вычислить линейные адреса сегментов TSS наших двух задач
        pop     eax
        push    eax
        add     eax,offset TSS_0
        mov     word ptr GDT_TSS0+2,ax
        shr     eax,16
        mov     byte ptr GDT_TSS0+4,al
        pop     eax
        add     eax,offset TSS_1
        mov     word ptr GDT_TSS1+2,ax
        shr     eax,16
        mov     byte ptr GDT_TSS1+4,al
; открыть A20
        mov     al,2
        out     92h,al
; запретить прерывания
        cli
; запретить NMI
        in      al,70h
        or      al,80h
        out     70h,al
; переключиться в PM
        mov     eax,cr0
        or      al,1
        mov     cr0,eax
; загрузить CS
        db      66h
        db      0EAh
        dd      offset PM_entry
        dw      SEL_32bitCS

RM_return:
; переключиться в реальный режим RM
        mov     eax,cr0
        and     al,0FEh
        mov     cr0,eax

```